

Computer Architecture Lab 3 Report

CSIE B12902075 沈柏宏

CPU

The majority design of CPU is succeeded from lab1. For some modules that don't change a lot, I will just simply mention it functionality here. I will focus more on the new modules or new change in old modules in this lab. The logic of CPU itself will also be explained in the end after all its sub-modules.

- **PC:** At clock edge, forward the value of next_PC to cur_PC. The generation of next_PC will be explain in PC_update module later.
- **Instruction Memory:** Read instruction from memory based on the cur_PC.
- **Control:** Decode signals from the opcode from the instruction fetched. Apart from 7 signals in the textbook, I have added 4 more signals:
 - Branch: whether the instruction is a conditional branch instruction.
 - Jump: whether the instruction is a jump (unconditional branch) instruction.
 - JumpR: whether the instruction is a jalr instruction.
 - ALUSrc2: The original ALUSrc signal becomes ALUSrc1. This is how these 2 signal affect ALU operand sources.

ALUSrc1	0	1
operand 1	reg data 1	pc

ALUSrc2	0	1
operand 2	reg data 2	imm

- **Imm Gen:** As before, decode and extract, sign-extend the corresponding immediate field from each instruction.
- **Register:** As before.
- **ALU control:** As before, decode which ALU operation is used based on ALU_op signal, funct3 and funct7.
- **ALU:** As before, execute the operation given by ALU control.
- **Branch Unit:** The module is new in this lab. It takes 3 inputs: 2 data read from register and the funct3 field of instruction. It outputs one signal: branch_taken. The inner logic is simple, use funct3 to determine which branch instruction it is, regardless of the opcode, which is decoded by control module to a Branch signal. The main and only function of branch unit module is to do the comparison(=, $\neq$, $<$, $\geq$).
- **PC update:** When branch and jump instruction is added, the PC updating logic becomes more complicated, so this module is added for help.

This module takes several inputs including:

- 3 possible pc sources: `cur_PC`, `imm`, `reg_data1`
- 5 control signals: `Jump`, `JumpR`, `Branch`, `branch_taken`, `Stall`.

With these inputs, we can determine the output `next_PC` by the order of these rules: `stall` → `JumpR` → `Jump` → `Branch` and `branch_taken` → normal case. We don't explicitly write the value they are updated to, as there is no difference from just writing code here.

- **Write Back:** The register write back logic is to first see if this is a jump instruction by the signal `Jump`. If it is, we write back `cur_PC + 4`. And then, if this is a memory read instruction, we write back the data read from memory. In other normal cases, we write back the result of ALU.

Cache

Cache Type

I implemented a 2-way skewed-associative cache. The concept is similar to original 2-way set-associative cache. The difference is we use two different indexing methods in 2 ways to reduce the number of conflicts. In `way0`, we use the original index bits, while in `way1`, we reversible hash for the index bits as new index. The reversible hash we choose here is to simply xor with the last few bits of tag bits.

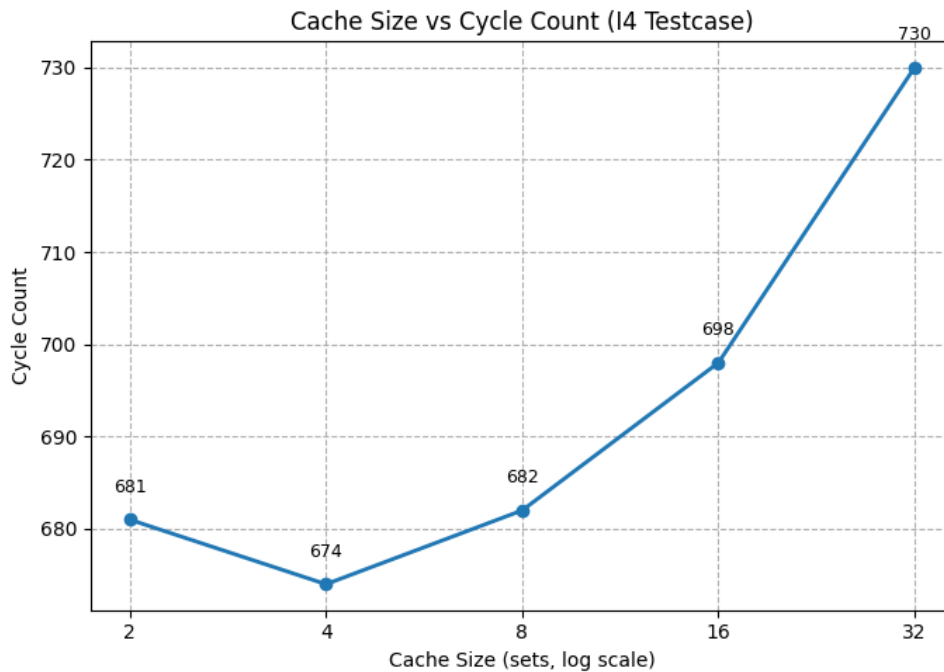
Configurations and Policies

- **write policy:** The cache uses a write-back + write-allocate policy. On a write hit, only the cache line is updated and marked dirty; memory is not immediately written. On a write miss, the block is first fetched from memory (allocate), updated in-cache, and marked dirty. This minimizes memory traffic and improves performance for workloads with temporal locality.
- **replacement policy:** Each set maintains a single LRU bit. When both ways are valid, the LRU bit determines the victim: `0` → evict `way0`, `1` → evict `way1`. The LRU bit is updated on every hit or refill. Combined with skewed indexing, this aims to reduce conflict misses, but I did not measure its hit rate against a standard 2-way LRU cache.
- **draining:** When the CPU issues its finish signal, the cache enters a drain mode. It iterates over all sets and both ways, writing back every dirty line to memory. This ensures the memory image is fully consistent before simulation ends. After all lines are clean, the cache transitions to a final `DONE` state and asserts `o_cache_finish` to the CPU.
- **alignment:** To support memory systems that start at a nonzero base offset, the cache clamps load addresses below `i_offset` and realigns incoming 128-bit data if the block boundary is shifted. Depending on the offset's low 2 bits, the fetched data may be shifted by 32, 64, or 96 bits to maintain correct word ordering inside the cache block. This is a laboratory-specific adaptation not found in standard L1 caches.
- **Finite State Machine:** The FSM includes five states:
 - **S_IDLE** – normal operation; detect hits/misses.
 - **S_ALLOC** – load block from memory on a miss.
 - **S_WB** – write back a dirty victim block.
 - **S_DRAIN** – sequentially write back all dirty blocks when finishing.

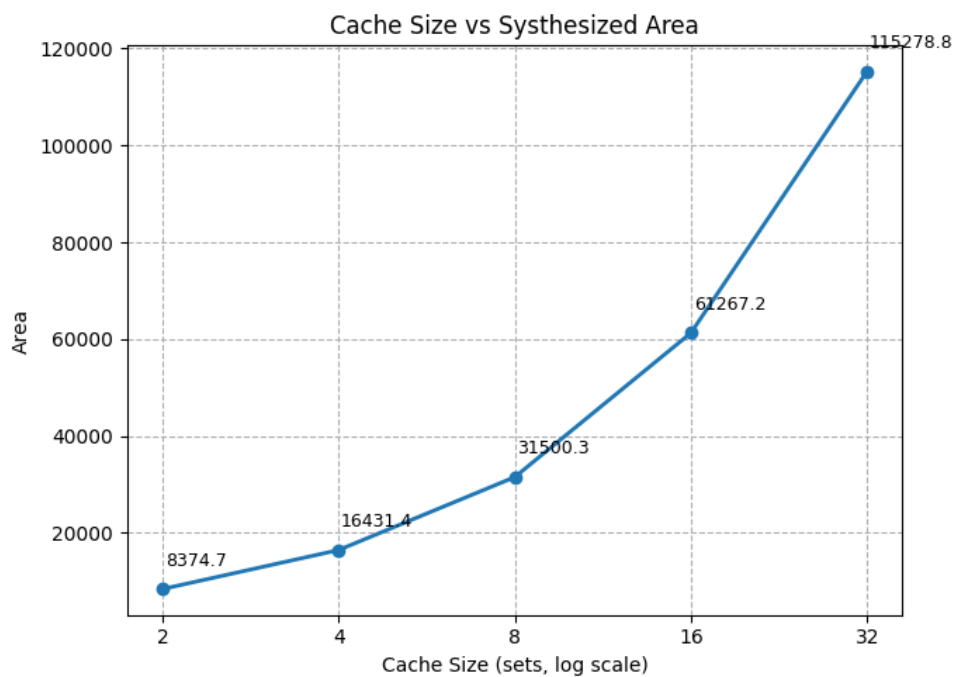
- **S_DONE** – all lines are clean; report cache finish to CPU.

This FSM ensures correct sequencing of eviction, refill, and termination behavior.

Cache Size vs. Execution Cycles



Cache Size vs. Cache Area



Execution Results

```

=== Running I4 ===
VCD info: dumpfile Final.vcd opened for output.
-----

START!!! I4 Simulation Start .....

-----

=====

Success!
The test result is .....PASS :)
Total execution cycle :      681
=====

```

(a) size = 2

```

=== Running I4 ===
VCD info: dumpfile Final.vcd opened for output.
-----

START!!! I4 Simulation Start .....

-----

=====

Success!
The test result is .....PASS :)
Total execution cycle :      674
=====

```

(b) size = 4

```

=== Running I4 ===
VCD info: dumpfile Final.vcd opened for output.
-----

START!!! I4 Simulation Start .....

-----

=====

Success!
The test result is .....PASS :)
Total execution cycle :      682
=====

```

(c) size = 8

```

=== Running I4 ===
VCD info: dumpfile Final.vcd opened for output.
-----

START!!! I4 Simulation Start .....

-----

=====

Success!
The test result is .....PASS :)
Total execution cycle :      698
=====

```

(d) size = 16

```

=== Running I4 ===
VCD info: dumpfile Final.vcd opened for output.
-----

START!!! I4 Simulation Start .....

-----

=====

Success!
The test result is .....PASS :)
Total execution cycle :      730
=====

```

(e) size = 32

LLM Support

In CPU implementation, I use LLMs to decode the instructions from binary, and construct their mapping to control signals. I also use them for some debug advice after I designed the datapath, as well as the final formatting.

In cache implementation, I first consulted them for some examples of modern fancy caches. After building the framework by myself, I asked them about some detail revision. Lastly, I at times use them to analyze the problem from the waveform.

Synthesizability / latch check screenshot

```
=====
=== Module Areas ===
ALU: 4357.080
ALU_Control: 40.698
Branch_Unit: 135.660
CPU: 5666.598
Control: 26.600
Imm_Gen: 75.810
MUX32: 59.584
PC: 170.240
PC_Update: 591.318
=====
Total area: 11123.588

=== Latch Statistics (PROC_DLATCH pass) ===
No latch inferred count : 17
Latch inferred count    : 0

=====
Cache Synthesis Log Analysis
=====
=== Module Areas ===
Cache: 16431.352
=====
Total area: 16431.352

=== Latch Statistics (PROC_DLATCH pass) ===
No latch inferred count : 44
Latch inferred count    : 0
```

Development environment

- OS: WSL2, Ubuntu 22.04
- Compiler: Yosys 0.9 (git sha1 1979e0b)
- IDE: Visual Studio Code 1.106.2